

DevLabs: Builder Profiles, Embeddings, Matching & Founder Search

Technical context document for handoff to another model or engineer.
Repository: devlabs/website · Generated: June 4, 2026

Executive summary: Talent OS builder profiles are **structured MongoDB documents**, not vector-indexed for founder matching. Founder search loads up to 2,000 eligible builders and ranks them with a **deterministic weighted heuristic** in `rankBuildersForOpportunity`. Vector embeddings exist for a **separate** club/admin resume pipeline. Supermemory provides **chat memory** for agents, not search retrieval.

Table of contents

- 1. System comparison (embeddings vs matching)
- 2. Builder profile — data model & usage
- 3. Embeddings in this repo (what exists vs talent OS)
- 4. Founder search & talent discovery flow
- 5. Matching algorithm (deterministic)
- 6. AI agent role (orchestration vs discovery)
- 7. Identity linking & practical implications
- 8. Key source files

1. System comparison

System	Embeddings?	Used for founder matching?
Talent OS (BuilderProfile + ProjectRecord)	No	Yes — rule-based scoring
Mongo Atlas vector collection (embeddings)	Yes (Cloudflare @cf/baai/bge-small-en-v1.5)	No — admin/club resume search only
Supermemory	Semantic memory API (not builder graph index)	No — founder/builder chat context only

2. Builder profile — data model & usage

2.1 Storage (structured Mongo, not vectors)

Profiles live in `BuilderProfile` (`src/models/talent/BuilderProfile.ts`) with nested `profileCompletion`, `profileQuality`, `links`, `availability`, `hiringIntent`, `verificationStatus`, `visibilityStatus`.

Proof-of-work lives in `ProjectRecord` (`src/models/talent/ProjectRecord.ts`), keyed by `builderId`: `projectName`, `description`, `techStack`, `builderContribution`, `links`, `verificationStatus`, `source`.

2.2 How profiles are created / updated

Path	What happens
Login / first agent call	<code>resolveAuthedBuilder</code> auto-creates minimal <code>BuilderProfile</code> if missing (<code>actionsHandler.ts</code>)
Builder agent (<code>builder_chat</code>)	Tools: <code>get_builder_profile</code> , <code>update_availability</code> , <code>update_links</code> , <code>update_profile_basics</code> , <code>import_project</code> , <code>evaluate_profile</code> , <code>fetch_intros/trials/calls/messages</code>
Resume upload (<code>/api/agent/upload-resume</code>)	PDF → Cloudinary URL + <code>parseAndExtractResume</code> (LLM JSON → Mongo). No vector index.
Bootstrap (<code>/api/talent/bootstrap-builders</code>)	Copies legacy User/Application into <code>BuilderProfile</code>
Devpost import API	Separate talent import path
Profile scores	<code>computeBuilderScores</code> (<code>matching.ts</code>); <code>evaluateBuilderProfileQuality</code> optional LLM (<code>profileQuality.ts</code>)

2.3 Builder resume parsing (Mongo only)

File: `src/lib/talent/resumeParser.ts`

1. pdf-parse extracts text
2. OpenRouter LLM returns JSON: headline, bio, skills, links, projects
3. Updates `BuilderProfile` (only empty fields for headline/bio/links)
4. Upserts `ProjectRecord` rows with source `resume_parser`

Separate pipeline: Club `/api/user/uploadResume` → `upsertResume` in `resumeProcessor.ts` → chunks → `getVectorStore().addDocuments()` with `user_id` metadata. That does NOT update Talent OS `BuilderProfile` for matching.

2.4 Where profiles are read

- **Builder OS:** get_builder_dashboard, builder_chat tools, profile/home tabs
- **Founder OS:** Shortlist, kanban, FounderCandidateDrawer, intro/trial flows
- **Matching:** Full scan of eligible builders at search time

2.5 Profile processing (quality, not embeddings)

1. **computeBuilderScores** (matching.ts): profileScore, proofScore, matchScore (readiness), missingItems, eligibility (not_eligible | eligible | priority)
2. **evaluateBuilderProfileQuality:** deterministic baseline; if OPENROUTER_API_KEY set, LLM enhancement stored on builder.profileQuality (source: llm | deterministic)
3. **Supermemory:** addBuilderMemory / addFounderMemory after agent writes — text snippets for chat, container tags builder-{id}, founder-{id}

There is no code path that embeds BuilderProfile or ProjectRecord into the Atlas vector index used for founder matching.

3. Embeddings in this repo

3.1 Vector stack (club / admin only)

File: src/lib/vectorStore.ts

- Embeddings: CloudflareEmbeddings, model CLOUDFLARE_EMBEDDING_MODEL default `@cf/baai/bge-small-en-v1.5`
- Store: MongoDBAtlasVectorSearch on collection MONGO_DB_COLLECTION (default `embeddings`)
- Index: MONGO_VECTOR_INDEX (default `vector_embeddings`), keys text + embedding

3.2 Ingestion

resumeProcessor.upsertResume: chunk resume ~100 tokens, metadata user_id, major, email, tags → vectorStore.addDocuments. Triggered from /api/user/uploadResume only.

3.3 Query

- POST /api/search/vector-only — similarity search, aggregate by user_id
- POST /api/search/rag — vector retrieval + LLM answer
- AdminDashboard.tsx — vector vs RAG modes

Not used by founder-portal or FounderOSDashboard.

3.4 Other embedding-related code

- api/embeddings/client.ts — MiniLM embedder for client company text
- api/search/matchClients.ts — cosine similarity client vs user vectors

3.5 Supermemory (agent memory)

File: src/lib/talent/supermemory.ts

- getBuilderMemoryProfile / getFounderMemoryProfile — profile API with optional query
- Injected into builder_chat / founder_chat system prompts (static slice 5, dynamic slice 3)
- Written on role brief create/update, profile updates, imports, intros

Not used for similarity search against opportunities.

4. Founder search & talent discovery

4.1 End-to-end flow

```
Founder UI (FounderRoleIntakeChat, FounderOSDashboard, RoleBriefEditor) → founder_chat  
(brief intake) OR direct API → Opportunity document in Mongo (draft → shortlisted) →  
run_builder_search OR founder_chat tool run_search → Load BuilderProfile  
(verificationStatus != rejected, visibilityStatus != hidden, limit 2000) → Load all  
ProjectRecords for those builderIds → rankBuildersForOpportunity(builders,  
projectsByBuilder, opportunity, limit 12) → Upsert MatchRecord per candidate; upsert  
Shortlist → Founder kanban / candidate drawer (buildFullCandidatesForShortlist when  
unlocked)
```

4.2 Gate before search

canRunPreviewAnyway (founderSearchQuality.ts) requires:

- roleTitle (not placeholder "New role")
- builderWillDo OR startupSummary
- skillsNeeded (non-empty array)

Full REQUIRED_FIELD_CHECKS also include workType/hireType, timeline, location — used for UI missing-field prompts, not the minimal search gate.

4.3 run_builder_search (actionsHandler.ts)

```
builders = BuilderProfile.find({ verificationStatus: { $ne: 'rejected' },  
visibilityStatus: { $ne: 'hidden' } }).limit(2000)  
allProjects = ProjectRecord.find({ builderId: { $in: builderIds } })  
ranked = rankBuildersForOpportunity(builders, projectsByBuilder, oppPlain, 12)  
→ MatchRecord.findOneAndUpdate per top match
```

```
→ Shortlist.findOneAndUpdate (unlocked: true in current code)
→ opportunity.status = 'shortlisted'
```

4.4 UI triggers

- FounderRoleIntakeChat: founder_chat; triggerSearch calls action run_builder_search (not HTTP action name run_search)
- Founder agent tool run_search: same ranking, preview count 8 in tool path
- rerun_search: preserves in-pipeline builders not in new top ranks

5. Matching algorithm (deterministic)

Core: `rankBuildersForOpportunity` in `src/lib/talent/builderSearch.ts`

5.1 Component scores (each 0–1)

Component	Weight in matchScore	Computation
skillFit	30%	Fuzzy skill match: skillsNeeded, niceToHaveSkills, roleTitle tokens vs headline, bio, rolePreference, profileQuality strengths, all project fields. SKILL_ALIASES (e.g. goLang/go, gcp/google cloud). Default 0.55 if no requirements.
proofRelevance	25%	Prefer verified projects; score project count, contribution length, tech stack presence; fallback to profileCompletion.proofScore
availabilityFit	15%	availableNow, hoursPerWeek, remotePreference vs opportunity locationPreference
workTypeFit	10%	preferredWorkType vs opportunity workType (aliases: full_time, internship, paid_sprint, etc.)
domainRelevance	10%	Token overlap: startupSummary, industry, builderWillDo, company vs project descriptions
profileQuality	10%	profileQuality.overallScore + founderClarity.score blended

5.2 Composite scores

```
matchScore = round(weighted sum of components * 100)
profileStrength = profileStrengthScore(builder, projects) // separate formula, links +
projects + quality
rankingStrength = round(matchScore * 0.7 + profileStrength * 0.3)
```

5.3 Hard filter

If opportunity has specific requirements (skills, roleType, or roleTitle) AND skillFit < 0.25 AND domainRelevance < 0.2 → builder excluded from results.

5.4 Labels & sort

- matchScore ≥ 80 → Strong Match; ≥ 65 → Good Match; else Possible Match
- Sort: rankingStrength desc, profileStrength, matchScore, skillFit, proofRelevance
- Return top 12; toAnonymousCandidates often slices 6–8 for preview cards

5.5 Explainability (templates, no LLM in ranker)

- whyTheyMatch — sentences from component thresholds
- proofSummary, availabilitySummary — buildAnonymousProofSummary, buildAvailabilitySummary
- signals — skillMatch, proofOfWork, availability, startupReadiness as low|medium|high

Persisted: MatchRecord.matchScore, profileStrength, matchLabel, reasoning, signalScores; Shortlist.candidates[] anonymous cards.

5.6 Post-search pipeline (not re-ranked by AI)

Founder actions: request_intro, update_candidate_status, schedule_call, send_trial_project, hire_builder, reject_builder, etc. MatchRecord.status progresses: generated → intro_requested → interviewing → trial → hired / rejected.

6. AI agent role

Step	AI?	Mechanism
Collect role brief	Yes	founder_chat: create_role_brief, update_role_brief; sanitize via founderRoleBrief.ts
Remember preferences	Optional	Supermemory + loadFounderStartupContext from last Opportunity
Find builders	No (ranking)	rankBuildersForOpportunity
Explain shortlist / candidates	Yes	get_shortlist, get_candidate_details tools
Request intro	Yes (guarded)	request_intro tool; creates IntroRequest, notifyBuilderIntroReceived

Improve builder profile	Yes	builder_chat tools; evaluate_profile uses LLM quality eval
-------------------------	-----	--

Talent discovery for founders = rule-based scan over structured profiles. Agent value = intake, brief hygiene, triggering search, candidate narrative — not vector similarity.

6.1 Related agents (separate doc)

Full agent architecture (builder_chat / founder_chat tools, OpenRouter params): see DevLabs-AI-Agent-Architecture.pdf in same docs folder.

7. Identity linking

deterministicBuilderMatch (matching.ts): email exact → full name unique → first+last unique. Used for imports/claims, not opportunity ranking.

8. Practical implications for another model

- 1. To improve founder match quality: enrich structured fields (skills, builderContribution, availability) and tune builderSearch.ts weights/aliases — not embedding models.
- 2. Admin vector search will NOT find Talent OS builders unless a new pipeline embeds BuilderProfile+projects into an index.
- 3. Two resume pipelines: club User → vector chunks; Builder → LLM parse → Mongo only.
- 4. Search complexity: O(min(2000, builders)) × projects; in-memory, not Atlas Search on builders.
- 5. founder_chat run_search and run_builder_search share the same ranker; differ in preview count and HTTP/UI wiring.

9. Key source files

Concern	Path
Builder schema	src/models/talent/BuilderProfile.ts
Project schema	src/models/talent/ProjectRecord.ts
Shortlist / Match	src/models/talent/Shortlist.ts, MatchRecord.ts
Ranking	src/lib/talent/builderSearch.ts
Completion scores	src/lib/talent/matching.ts

Search API	src/lib/agent/actionsHandler.ts (run_builder_search, founder_chat run_search)
Brief gates	src/lib/talent/founderSearchQuality.ts
Vector store	src/lib/vectorStore.ts, src/lib/resumeProcessor.ts
Builder resume parse	src/lib/talent/resumeParser.ts
Agent memory	src/lib/talent/supermemory.ts
Candidate UI hydrate	src/lib/talent/founderCandidate.ts
Founder search UI	src/components/founder/FounderRoleIntakeChat.tsx
Vector search API	src/pages/api/search/vector-only.ts, rag.ts

9.1 Environment variables (relevant)

- OPENROUTER_API_KEY, OPENROUTER_MODEL_CHAT — LLM (agents, resume parse, profile quality)
- SUPERMEMORY_API_KEY — optional agent memory
- MONGODB_URI / ADMIN_MONGO_URI — BuilderProfile, Opportunity, embeddings collection
- CLOUDFLARE_ACCOUNT_ID, CLOUDFLARE_API_TOKEN, CLOUDFLARE_EMBEDDING_MODEL — vector pipeline only
- MONGO_VECTOR_INDEX, MONGO_DB_COLLECTION — vector collection config

9.2 API actions reference (founder talent path)

```
POST /api/agent/actions
{ "action": "founder_chat", "payload": { "message", "opportunityId?", "history" } }
{ "action": "run_builder_search", "payload": { "opportunityId" } }
{ "action": "rerun_search", "payload": { "opportunityId" } }
{ "action": "unlock_shortlist", ... }
{ "action": "request_intro", ... }
{ "action": "get_founder_dashboard", ... }
```